

MACHINE LEARNING LAB MANUAL.

6TH SEM BCA

Sl.No	Name of the Program	Date
1	Install and set up Python and essential libraries like NumPy and pandas	8.3.25
2	Introduce scikit-learn as a machine learning library.	8.3.25
3	Install and set up scikit-learn and other necessary tools	8.3.25
4	Write a program to Load and explore the dataset of .CSV and excel files using pandas.	10.3.25
5	Write a program to Visualize the dataset to gain insights using Matplotlib or Seaborn by plotting scatter plots, and bar charts.	18.3.25
6	Write a program to Handle missing data, encode categorical variables, and perform feature scaling	25.3.25
7	Write a program to perform House price predict using Linear Regression	9.4.25
8	Write a program to Binary Classification using Logistic Regression	9.4.25
9	Write a program to implement a k-Nearest Neighbours (k-NN) classifier using scikit-learn and Train the classifier on the dataset and evaluate its performance.	15.4.25
10	Write a program to implement a linear regression model for regression tasks and Train the model on a dataset with continuous target variables.	17.4.25
11	Write a program to implement a decision tree classifier using scikit-learn and visualize the decision tree and understand its splits.	22.4.25
12	Write a program to implement K-Means Clustering and Visualize the Cluster	29.4.25
13	Write a program to show matching z score Values	6.5.25
14	Write a Program to handle outliers	6.5.25

Program 1 Install and set up Python and essential libraries like NumPy and pandas.

a)Installing Python:

i)Download Python:-

Go to the official Python website download the latest version suitable for your operating system (Windows, macOS, or Linux).

ii)Install Python:-

For Windows: Download python software and install it.

iii)Verify Installation:-

- Open a command prompt (Windows)
- `python --version`

b)Pip upgrade: Using pip:

- Pip is Python's package manager.
- It usually comes installed with Python.
- Open a terminal/command prompt.

Py –m install pip –upgrade pip

c) Install NumPy :

pip install numpy

press Enter. This command will download and install NumPy.

d)Install pandas:

pip install pandas

press Enter. This command will download and install pandas.

e) Verify installations:

Open a IDLE

 *program1.py - C:/niha/program1.py (3.12.2)*

File Edit Format Run Options Window Help

```
import numpy
import pandas
print(numpy.__version__)
print(pandas.__version__)
```

OUTPUT:

1.26.3
2.2.0

Program 2 Introduce scikit-learn as a machine learning library.

- Scikit-learn (Sklearn) is the most useful and robust library for machine learning in Python.
- It provides a selection of efficient tools for machine learning and statistical modeling including classification, regression, clustering and dimensionality reduction via a consistent interface in Python.
- This library, which is largely written in Python, is built upon NumPy, pandas, SciPy and Matplotlib.

Installation

If you already installed NumPy and Scipy, the following are the two easiest ways to install scikit-learn –

Using pip

The following command can be used to install scikit-learn via pip

pip install scikit-learn

Features

Rather than focusing on loading, manipulating and summarizing data, Scikit-learn library is focused on modeling the data. Some of the most popular groups of models provided by Sklearn are as follows.

Supervised Learning algorithms – Almost all the popular supervised learning

algorithms, like Linear Regression, Support Vector Machine (SVM), Decision Tree etc., are the part of scikit-learn.

Unsupervised Learning algorithms – On the other hand, it also has all the popular unsupervised learning algorithms from clustering, factor analysis, PCA (Principal Component Analysis) to unsupervised neural networks.

Clustering – This model is used for grouping unlabeled data.

Cross Validation – It is used to check the accuracy of supervised models on unseen data.

Program 3. Install and set up scikit-learn and other necessary tools.

Pip upgrade:

Using pip:

- Pip is Python's package manager.
- It usually comes installed with Python.
- Open a terminal/command prompt.

Py -m install pip - -upgrade pip

a) Install

NumPy :- pip

install numpy

press Enter. This command will download and install NumPy.

b)Install

pandas:- pip

install pandas

press Enter. This command will download and install pandas.

c) Install

matplotlib:- pip

install matplotlib

press Enter. This command will download and install matplotlib.

d)Install scipy:-

pip install scipy press Enter. This command will download and install scipy

e) Install scikit-

learn(sklearn):- pip install

scikit-learn

press Enter. This command will download and install scikit-learn

Verify installations:

 program3.py - C:/niha/program3.py (3.12.2)

File Edit Format Run Options Window Help

```
import sklearn
import numpy
import pandas
import matplotlib
print(sklearn.__version__)
print(numpy.__version__)
print(pandas.__version__)
print(matplotlib.__version__)
```

OUTPUT:

```
1.4.2
1.26.3
2.2.0
3.8.2
```

Program 4 Write a program to Load and explore the dataset of .CSV and excel files using pandas.

Step 1: Creating CSV and Excel Files with Dummy Data

1. Create CSV File:

- Open a text editor like Notepad or any other code editor.
- Enter the following data:

Name, Age, Score

Srikanth, 28, 85

Snigdha, 22, 78

Mary, 31, 92

- Save this file as **sample_data.csv** in the **C:\ML_Projects** directory.

2. Create Excel File:

- Use Microsoft Excel or Google Sheets to create this file.
- Enter the following data:

Course	Name	Sem
BCA	Rajesh	1
BCA	Ramesh	2
BCOM	Swati	1
BCOM	Florina	3
BBA	Pooja	2
BBA	Raghu	4

- Save this file as **sample_data.xlsx** in the **C:\ML_Projects** directory.

Step 2: Python Code to Load and Explore the Data

```
import pandas as pd
# Define the file paths
csv_file_path = 'C:\\ML_Projects\\sample_data.csv' excel_file_path =
'C:\\ML_Projects\\sample_data.xlsx'
# Load the CSV file
data_csv = pd.read_csv(csv_file_path)
```

```
print ("CSV File Data:")
print (data_csv)
#Load the Excel file
data_excel = pd.read_excel (excel_file_path)
print("\nExcel File Data:")
print (data_excel)
# Basic Data Exploration print("\nData Descriptions:") print("CSV Data Description:")
print (data_csv.describe())
print("\nExcel Data Description:") print (data_excel.describe()) # Displaying data types
print("\nData Types in CSV File:") print (data_csv.dtypes) |
print("\nData Types in Excel File:")
print (data_excel.dtypes)
```

Output

CSV File Data:

Name: Srikanth, Age: 28, Score: 85
Name: Snigdha, Age: 22, Score: 78
Name: Mary, Age: 31, Score: 92

Excel File Data:

Name: Rajesh, Course: BCA, Sem: 1
Name: Ramesh, Course: BCA, Sem: 2
Name: Swati, Course: BCOM, Sem: 1
Name: Florina, Course: BCOM, Sem: 3
Name: Pooja, Course: BBA, Sem: 2
Name: Raghu, Course: BBA, Sem: 4

Data Descriptions:

CSV Data Description:

Count: Age: 3, Score: 3
Mean: Age: 27.00, Score: 85.00
Standard Deviation: Age: 4.58, Score: 7.80
Minimum: Age: 22.00, Score: 78.00
25th Percentile: Age: 25.00, Score: 81.50
50th Percentile (Median): Age: 28.00, Score: 85.90

75th Percentile: Age: 29.50, Score: 88.50
Maximum: Age: 31.00, Score: 92.00

Data Types in CSV File:

Name: object
Age: int64
Score: int64

Data Types in Excel File:

Name: object
Course: object
Sem: int64
dtype: object

Program 5 Write a program to Visualize the dataset to gain insights using Matplotlib or Seaborn by plotting scatter plots, and bar charts.

```
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import pandas as pd

# Step 1: Create a synthetic dataset
np.random.seed(42)

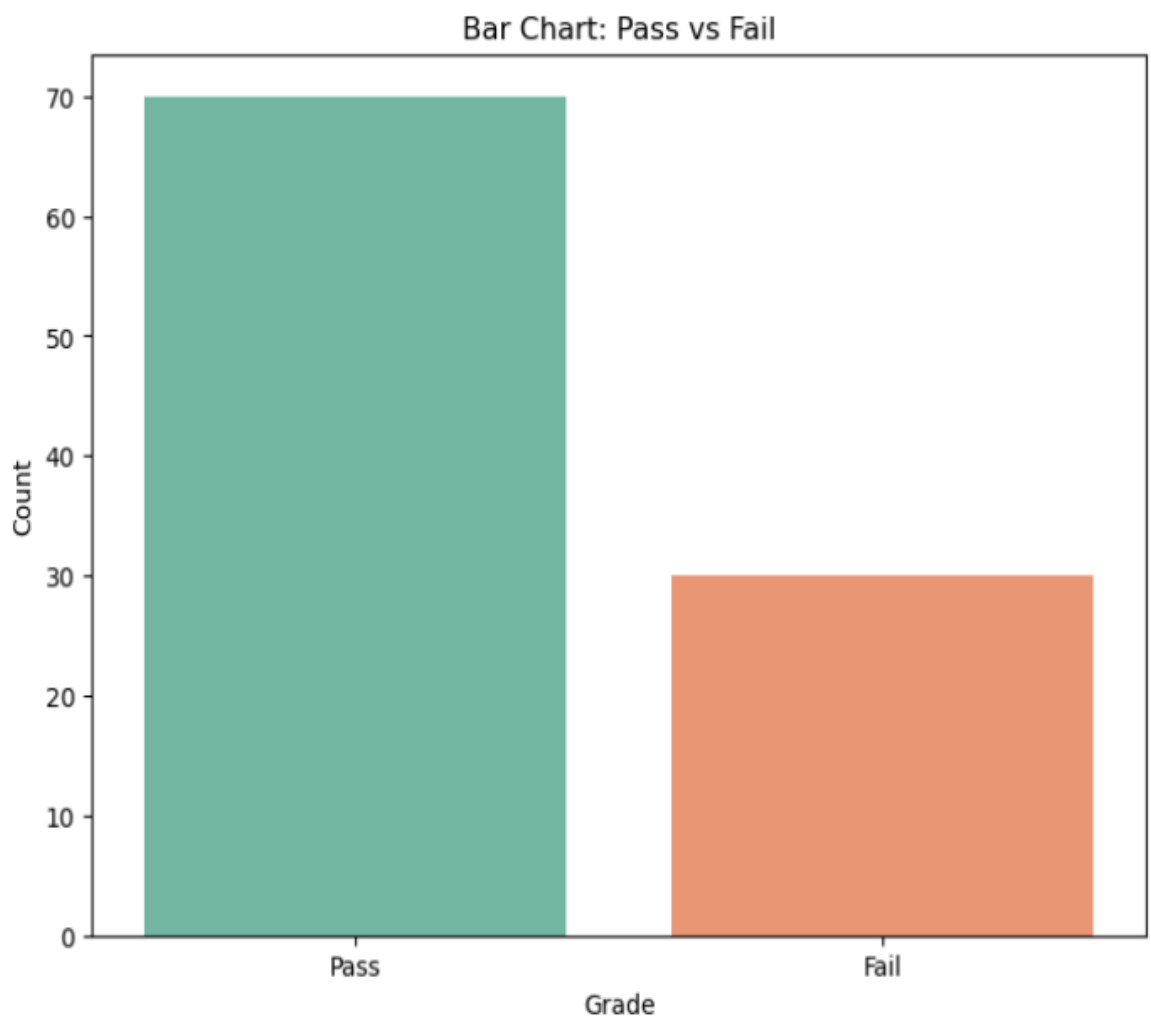
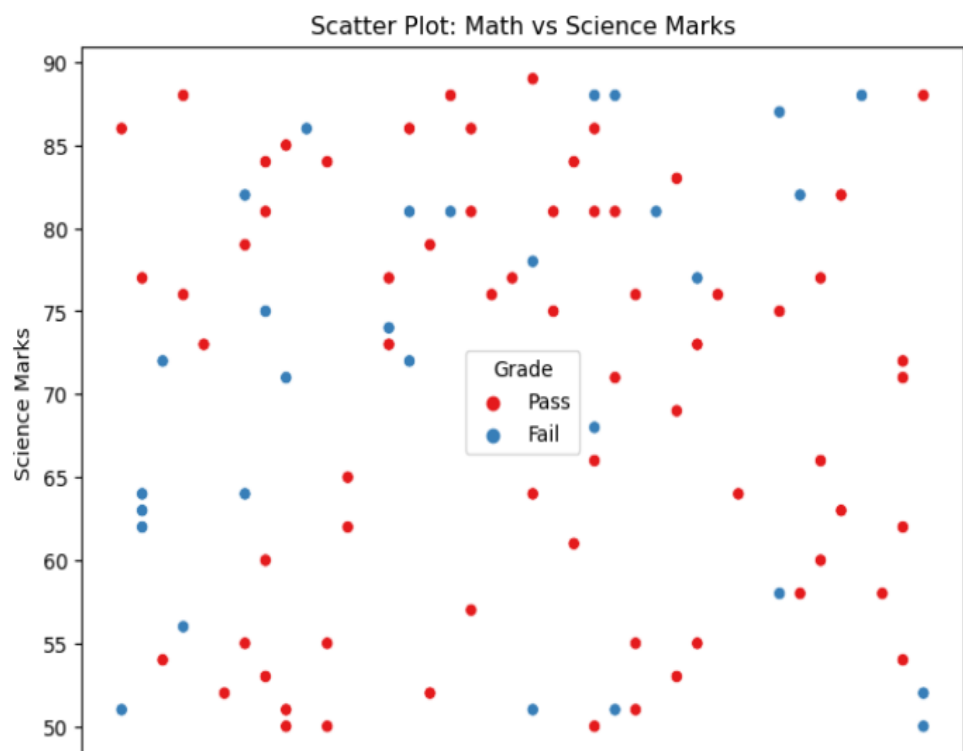
# Generate marks data
math_marks = np.random.randint(50, 90, 100) # Math marks (50 to 90)
science_marks = np.random.randint(50, 90, 100) # Science marks (50 to 90)
grades = np.random.choice(['Pass', 'Fail'], size=100, p=[0.7, 0.3]) # Grades: Pass/Fail

# Combine into a DataFrame for better visualization
data = pd.DataFrame({
    'Math': math_marks,
    'Science': science_marks,
    'Grade': grades
})

# Step 2: Scatter plot to show the relationship between Math and Science marks
plt.figure(figsize=(8, 6))
sns.scatterplot(x='Math', y='Science', hue='Grade', data=data, palette='Set1')
plt.title('Scatter Plot: Math vs Science Marks')
plt.xlabel('Math Marks')
plt.ylabel('Science Marks')
plt.show()

# Step 3: Bar chart to visualize the distribution of grades (Pass/Fail)
plt.figure(figsize=(8, 6))
sns.countplot(x='Grade', data=data, palette='Set2')
plt.title('Bar Chart: Pass vs Fail')
plt.xlabel('Grade')
plt.ylabel('Count')
plt.show()
```


OUTPUT



Program 6 Write a program to Handle missing data, encode categorical variables, and perform feature scaling.

```
import pandas as pd
from sklearn.impute import SimpleImputer
data = {'Feature1': [10, 20, 30, None, 50],
        'Feature2': [5, None, 15, 20, 25]}
df = pd.DataFrame(data)
imputer = SimpleImputer(strategy="mean")
df_filled = pd.DataFrame(imputer.fit_transform(df), columns=df.columns)
print("Original DataFrame with Missing Values:")
print(df)
print("\nDataFrame after Handling Missing Values:")
print(df_filled)
```

Output

Original DataFrame with Missing Values:

	Feature1	Feature2
0	10.0	5.0
1	20.0	NaN
2	30.0	15.0
3	NaN	20.0
4	50.0	25.0

DataFrame after Handling Missing Values:

	Feature1	Feature2
0	10.0	5.00
1	20.0	16.25
2	30.0	15.00
3	27.5	20.00
4	50.0	25.00

Program 6 - B: Program for perform scaling

```
import numpy as np
from sklearn.preprocessing import StandardScaler
import matplotlib.pyplot as plt

# Step 1: Create a sample dataset (2 features, 5 data points)
data = np.array([[50, 200],
                 [60, 220],
                 [70, 240],
                 [80, 260],
                 [90, 280]])

# Step 2: Apply StandardScaler
scaler = StandardScaler()
scaled_data = scaler.fit_transform(data)

# Step 3: Print the original and scaled data
print("Original Data:")
print(data)

print("\nScaled Data:")
print(scaled_data)
plt.show()
```

Output

Original Data:

```
[[ 50 200]
 [ 60 220]
 [ 70 240]
 [ 80 260]
 [ 90 280]]
```

Scaled Data:

```
[[ -1.41421356 -1.41421356]
 [-0.70710678 -0.70710678]
 [ 0.          0.         ]
 [ 0.70710678  0.70710678]
 [ 1.41421356  1.41421356]]
```

Program 6 c) Write a Program to perform Categorical encoding

```
import pandas as pd
from sklearn.preprocessing import LabelEncoder

# Sample dataset
data = {'Color': ['Red', 'Blue', 'Green', 'Blue', 'Red'], 'Size': ['S', 'M', 'L', 'M', 'S']}
df = pd.DataFrame(data)

# Label Encoding for 'Color' column
df['Color_Encoded'] = LabelEncoder().fit_transform(df['Color'])

# One-Hot Encoding for 'Size' column
df = pd.get_dummies(df, columns=['Size'], drop_first=True)

# Output the result
print(df)
```

Output

	Color	Color_Encoded	Size_M	Size_S
0	Red	2	False	True
1	Blue	0	True	False
2	Green	1	False	False
3	Blue	0	True	False
4	Red	2	False	True

Program 7 Write a program to perform House price predict using Linear Regression

```
# Import necessary libraries
import numpy as np
from sklearn.linear_model import LinearRegression

# Given data sample
data = np.array([[1500, 5, 200],
                 [1200, 2, 180],
                 [1800, 4, 220],
                 [1000, 1, 160],
                 [1400, 3, 190]])

# Separate features (size and age) and target (price)
X = data[:, :2] # Features: size and age
y = data[:, 2]  # Target: price

# Create and train the linear regression model
model = LinearRegression()
model.fit(X, y)

# Take user inputs for size and age
new_size = float(input("Enter the size of the house: "))
new_age = float(input("Enter the age of the house: "))

# Predict the price for the new data point
predicted_price = model.predict([[new_size, new_age]])

# Print the predicted price
print("Predicted Price for a house with size {} sq. ft and age {} years: {:.2f} Lakhs".format(new_size, new_age, predicted_price[0]))
```

Program 8. Write a program to Binary Classification using Logistic Regression

```
import numpy as np
from sklearn.linear_model import LogisticRegression

# Binary Classification Example
# Data for binary classification (hours studied and pass/fail)
X_binary = np.array([[2], [4], [6], [8], [10]]) # Hours studied
y_binary = np.array([0, 0, 1, 1, 1]) # Pass (1) or Fail (0)

# Train a Logistic Regression model for binary classification
model_binary = LogisticRegression()
model_binary.fit(X_binary, y_binary)

# User input for prediction
hours_studied = float(input("Enter the number of hours studied: "))

# Predictions
binary_prediction = model_binary.predict_proba([[hours_studied]])

# Output
if binary_prediction[0][1] >= 0.5:
    print("Binary Classification - Predicted class: PASS")
else:
    print("Binary Classification - Predicted class: FAIL")
```

Output:

Enter the number of hours studied: 7
Binary Classification - Predicted class: PASS

Enter the number of hours studied: 1.5
Binary Classification - Predicted class: FAIL

Program 9 Write a program to implement a k-Nearest Neighbours (k-NN) classifier using scikit-learn and Train the classifier on the dataset and evaluate its performance.

```
import numpy as np

from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score

X=np.array([[80,75], [95,90], [60,50], [45,30], [30,40], [85,95], [70,60], [50,55],
[40,45], [60,70]])

y=np.array([1,1,0,0,0,1,1,0,0,1])

X_train, X_test, y_train, y_test = train_test_split(X, y,test_size=0.2,
random_state=42)

knn = KNeighborsClassifier(n_neighbors=3)

knn.fit(X_train,y_train)

y_pred = knn.predict(X_test)

accuracy = accuracy_score(y_test, y_pred)

print("accuracy on the test set: {:.2f}".format(accuracy))

exam_score1 = float(input("enter exam score 1:"))
exam_score2 = float(input("enter exam score 2:"))
user_input = np.array([[exam_score1, exam_score2]])
predicted_outcome = knn.predict(user_input)

if predicted_outcome[0] == 1:
    print("based on the exam score provided,the student is predicted to pass.")
else:
    print("based on the exam score provided,the student is predicted to fail.")
```

OUTPUT:

```
accuracy on the test set: 1.00
enter exam score 1: 60
enter exam score 2: 40
based on the exam score provided,the student is predicted to fail.
```

Program 10 Write a program to implement a linear regression model for regression tasks and Train the model on a dataset with continuous target variables.

```
import numpy as np
from sklearn.linear_model import LinearRegression
X = np.array([[1000, 2], [1500, 3], [1200, 2], [1800, 4], [900, 2], [2000, 3]])
y = np.array([300000, 400000, 350000, 500000, 280000, 450000])
model = LinearRegression()
model.fit(X, y)
size = float(input("enter the size of the house in sqft: "))
bedrooms = int(input("enter the number of bedrooms: "))
new_data = np.array([[size, bedrooms]])
predicted_price = model.predict(new_data)
print("predicted price for a house with size {} sqft and {} bedrooms:
Rs.{:.2f}".format(size, bedrooms, predicted_price[0]))
```

OUTPUT:

enter the size of the house in sqft: 1600

enter the number of bedrooms: 3

predicted price for a house with size 1600.0 sqft and 3 bedrooms: Rs.418163.93

Program 11 Write a program to implement a decision tree classifier using scikit-learn and visualize the decision tree and understand its splits.

```
from sklearn.tree import DecisionTreeClassifier, plot_tree
import matplotlib.pyplot as plt
import numpy as np

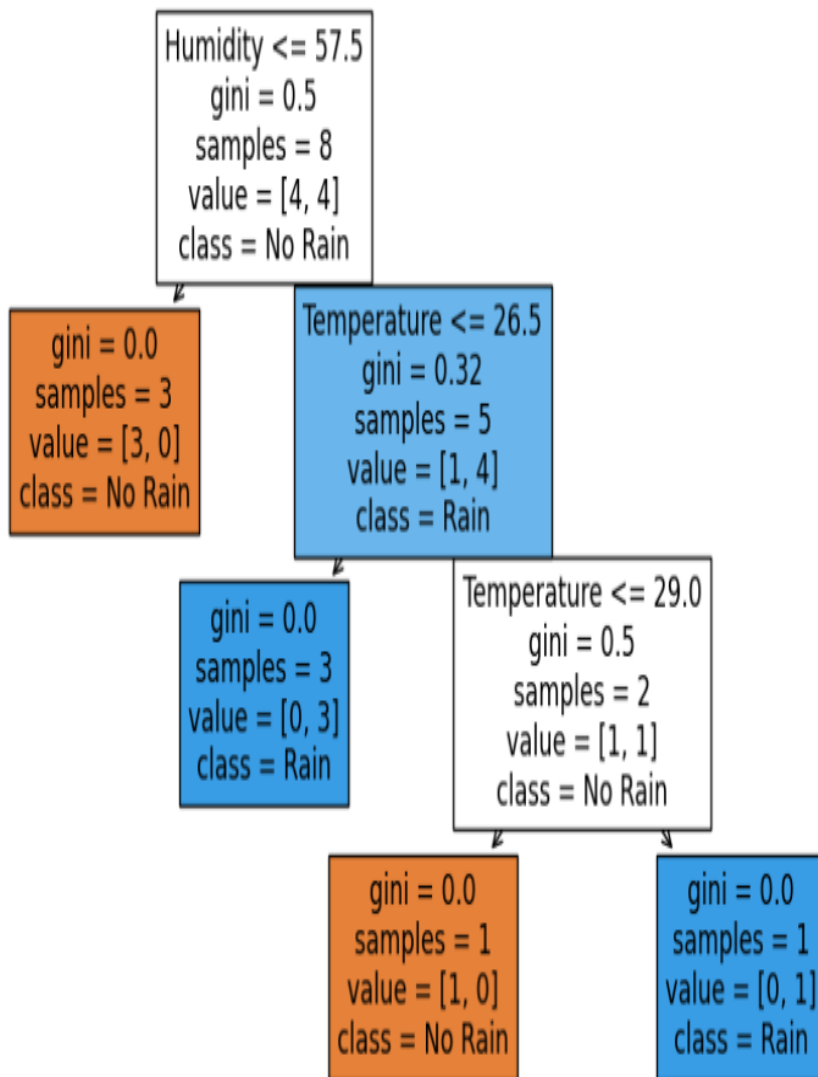
# Create a simple dataset (8 samples, 2 features)
# Features: [Temperature, Humidity]
X = np.array([
    [30, 70], # Sample 1
    [22, 60], # Sample 2
    [28, 65], # Sample 3
    [35, 55], # Sample 4
    [20, 80], # Sample 5
    [25, 75], # Sample 6
    [33, 50], # Sample 7
    [31, 55] # Sample 8
])

# Labels (0 = No Rain, 1 = Rain)
y = np.array([1, 1, 0, 0, 1, 1, 0, 0])

# Train Decision Tree
clf = DecisionTreeClassifier()
clf.fit(X, y)

# Visualize Decision Tree
plt.figure(figsize=(10, 6))
plot_tree(clf, filled=True, feature_names=["Temperature", "Humidity"],
class_names=["No Rain", "Rain"])
plt.title("Decision Tree on Weather Data")
plt.show()
```

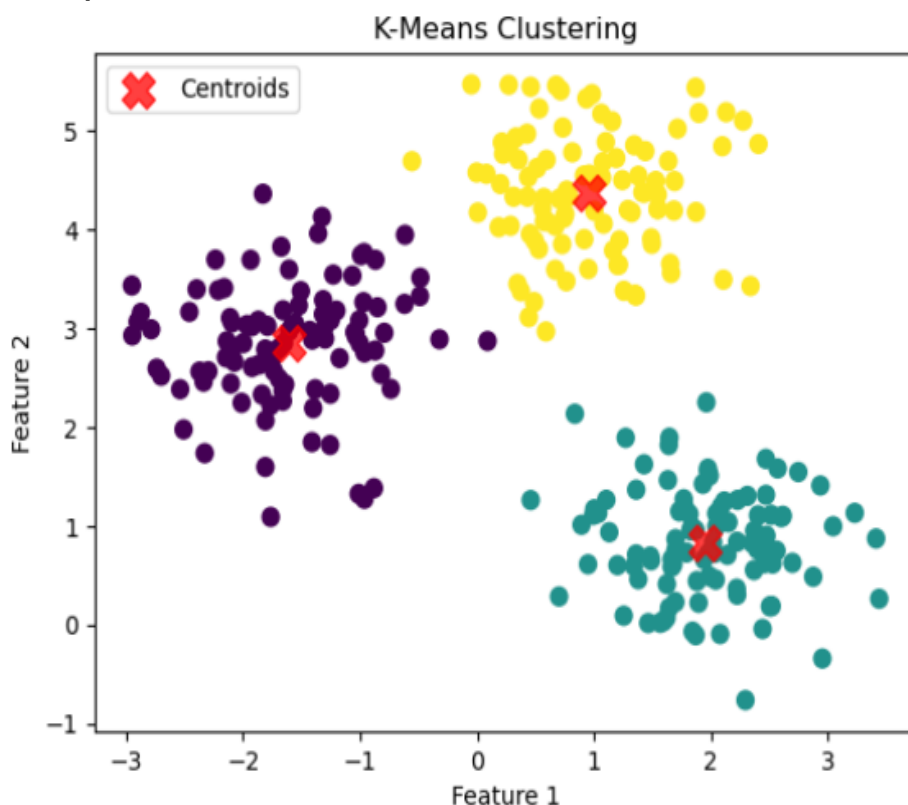
Decision Tree on Weather Data



Program 12: Write a program to implement K-Means Clustering and Visualize the Cluster.

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
from sklearn.datasets import make_blobs
X, y_true = make_blobs(n_samples=300, centers=3, cluster_std=0.60,
                        random_state=0)
kmeans = KMeans(n_clusters=3, random_state=0)
kmeans.fit(X)
y_kmeans = kmeans.predict(X)
plt.scatter(X[:, 0], X[:, 1], c=y_kmeans, s=50, cmap='viridis')
centers = kmeans.cluster_centers_
plt.scatter(centers[:, 0], centers[:, 1], c='red', s=200, alpha=0.75,
            marker='X', label='Centroids')
plt.title('K-Means Clustering')
plt.xlabel('Feature 1')
plt.ylabel('Feature 2')
plt.legend()
plt.show()
```

Output:



Program 13 Write a program to show matching z score Values

```
import numpy as np

# Sample data
data = [50, 60, 70, 80, 90, 100]

# Calculate Z-scores
mean = np.mean(data)
std_dev = np.std(data)
z_scores = [(x - mean) / std_dev for x in data]

# Input Z-score to match
input_z = float(input("Enter the Z-score to match: "))

# Find closest matching Z-score
closest_z_index = np.argmin(np.abs(np.array(z_scores) - input_z))
matching_value = data[closest_z_index]

# Output the result
print(f"Data point with Z-score closest to {input_z}: {matching_value}")
```

Output:

```
Enter the Z-score to match: 0.7
Data point with Z-score closest to 0.7: 90
```

Program 14 Write a Program to handle outliers

```
import numpy as np
from scipy.stats import zscore

# Data with outliers
data = np.random.normal(50, 10, 100).tolist() + [150, 200, 300]

# Calculate Z-scores and detect outliers ( $Z > 3$  or  $Z < -3$ )
outliers = np.abs(zscore(data)) > 3

# Print outliers
print("Outliers:", np.array(data)[outliers])
```

Outliers:
[200. 300.]